Assignment 06: Visitor Pattern - Advanced File System

Course: ShP-2216 Software Design Patterns | RK2 (15%) | Trimester 4

Student: Maksut Alibi
Group: SE2523
Format: Individual | Java 17 | Maven 3.9+

Repository: https://github.com/M1ska2021/Assignment6_Visitor
Build Tool: Maven (pom.xml + ./mvnw)
Verification: JUnit 5 (8/8 automated test targets passed)

1. Purpose and Architectural Design Choices

The goal of this assignment is to develop an in-memory hierarchical file system and implement the Visitor pattern to support three independent, extensible operations (TotalSizeVisitor, SearchVisitor, StatisticsVisitor) over the same composite tree.

1. Object Model & Double Dispatch:

* Element declares accept(Visitor) along with hierarchical getters/setters (getName(), getParent(), setParent()).
* FileNode represents files with non-negative byte sizes. DirectoryNode stores named children in arbitrary nesting.
* Double dispatch: (1) client invokes element.accept(visitor); (2) concrete node delegates visitor.visit(this). The compiler binds 'this' statically, routing execution to the correct visit overload without any instanceof branching.

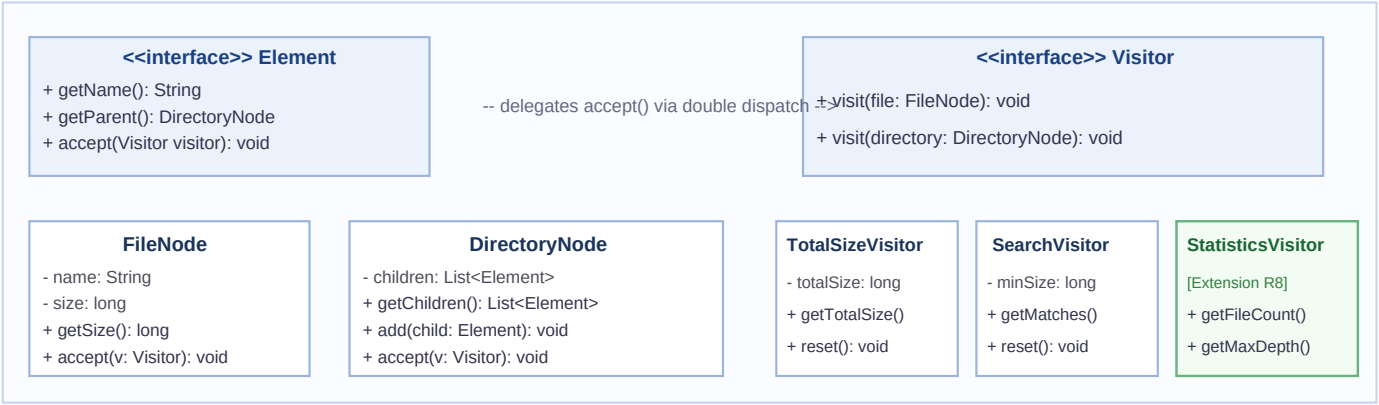
2. Traversal Ownership Decision (Visitor-Managed):

* Traversal is owned by Visitors inside visit(DirectoryNode), which iterate children and call child.accept(this).
* Rationale: Allows each visitor to customize its traversal state (e.g., path stack in SearchVisitor, depth counter in StatisticsVisitor) while preserving minimal accept(v) implementations in Element classes.

3. Controlled State & Tree Invariants:

* Idempotent reusability: activeDepth counter resets state automatically on root entry; explicit reset() is also available.
* Tree integrity: DirectoryNode.add() prevents negative file sizes, duplicate sibling names (IllegalArgumentException), and rejects multiple parent ownership and circular references (IllegalStateException) through parent-chain traversal.

2. UML Class Diagram & Architecture



Key Architectural Highlights & Invariant Defenses:

*** Negative Byte Size Defense:**	FileNode constructor validates 'size >= 0' and non-blank names, throwing IllegalArgumentException.
*** Duplicate Sibling Prevention:**	DirectoryNode.add checks existing child names before attachment, throwing IllegalArgumentException.
*** Single Parent Ownership:**	DirectoryNode.add verifies 'child.getParent() == null || child.getParent() == this' (IllegalStateException).
*** Cycle Detection via Ancestor Chain:**	DirectoryNode.add climbs the parent hierarchy to ensure child is not an ancestor (IllegalStateException).
*** Encapsulated Directory Collections:**	DirectoryNode.getChildren() returns Collections.unmodifiableList(), preventing external modification.
*** Root-Relative Deterministic Search:**	SearchVisitor tracks path stacks using ArrayDeque; results use '/' separator and are lexicographically sorted.
*** Depth & Metric Tracking:**	StatisticsVisitor records fileCount, directoryCount (including visited root), and maxDepth (root depth = 0).
*** Zero Runtime Type Discrimination:**	No instanceof, getClass(), reflection, or downcasting is used anywhere in the codebase.

3. Clean Code Practices (Concrete Justifications)

1. Meaningful Naming `Reference: TotalSizeVisitor.getTotalSize() and DirectoryNode.add(Element child)`

Justification: Method and class identifiers unambiguously express domain concepts without obscure abbreviations or cryptic shorthand, enabling immediate understanding of intent and contracts without requiring explanatory code comments.

2. Single Responsibility Principle (SRP) `Reference: FileNode / DirectoryNode vs. TotalSizeVisitor / SearchVisitor / Statisti`

Justification: Node classes are exclusively responsible for composite structural hierarchy and invariant enforcement, while analytical, search, and aggregation algorithms are isolated into dedicated visitor classes adhering to high cohesion.

3. Small, Focused Methods `Reference: FileNode.accept(Visitor visitor) and DirectoryNode.remove(Element child`

Justification: Methods perform a single atomic operation in 2-5 lines of code, minimizing cognitive complexity and maximizing testability and maintainability across edge conditions.

4. Encapsulation & Immutability `Reference: DirectoryNode.getChildren()`

Justification: Exposes internal child lists exclusively via Collections.unmodifiableList(children), preventing clients from bypassing validation or mutating directory state directly from outside the public API.

5. Elimination of Type-Checking Chains `Reference: FileNode.accept(Visitor visitor) and DirectoryNode.accept(Visitor visito`

Justification: Polymorphic double dispatch (visitor.visit(this)) routes execution to the exact overloaded visit method at compile/runtime, completely eliminating fragile 'if (node instanceof FileNode)' branching and downcasting.

4. Architectural Trade-offs & Extension Boundaries

A. Traversal Ownership Trade-off (Visitor-Managed vs. Element-Managed):

* Visitor-Managed Traversal (Selected): Traversal is controlled in visit(DirectoryNode). This grants individual visitors the autonomy to control search ordering, depth stacks, and path segments, keeping node elements lightweight and unburdened.
* Element-Managed Traversal (Alternative): Having DirectoryNode.accept recurse over children forces a single uniform traversal strategy onto all operations, depriving visitors of specialized pre/post-order actions or path tracking.

B. The Visitor Pattern Dual Trade-off (Operations vs. Element Types):

* Adding New Operations (Easy / Open-Closed): New visitor classes (e.g., StatisticsVisitor) are added without modifying existing classes (FileNode, DirectoryNode, Element, or other visitors). The extension boundary is the Visitor interface.
* Adding New Elements (Hard / Element Rigidity): Introducing a new element (e.g., SymlinkNode) breaks the Visitor interface, requiring a new visit(SymlinkNode) method and forcing modifications across every existing concrete visitor in the codebase.

5. Extensibility Demonstration (Requirement R8)

Extensibility Verification: Addition of StatisticsVisitor via Git Diff

The StatisticsVisitor was introduced independently in Git commit 3822609 without changing FileNode, DirectoryNode, or their accept methods. Below is the exact extension diff confirming zero modifications to existing classes:

```
diff --git a/src/main/java/kz/aitu/se2523/visitor/visitor/StatisticsVisitor.java
new file mode 100644
--- /dev/null
+++ b/src/main/java/kz/aitu/se2523/visitor/visitor/StatisticsVisitor.java
@@ -0,0 +1,64 @@
+package kz.aitu.se2523.visitor.visitor;
+
+public class StatisticsVisitor implements Visitor {
+    private int fileCount = 0;
+    private int directoryCount = 0;
+    private int maxDepth = 0;
+    private int currentDepth = 0;
+    public int getFileCount() { return fileCount; }
+    public int getDirectoryCount() { return directoryCount; }
+    public int getMaxDepth() { return maxDepth; }
+    @Override public void visit(FileNode file) { ... }
+    @Override public void visit(DirectoryNode directory) { ... }
+}
```

Conclusion: The extension boundary (Visitor interface) ensures that operational extensibility is 100% compliant with OCP.

6. Executable Program Output (Deterministic Demonstration)

Execution Console Log (kz.aitu.se2523.visitor.Main)

```
=== Visitor Pattern Demonstration ===
Root directory: portfolio

--- TotalSizeVisitor ---
Subtree size: 71570 bytes

--- SearchVisitor (.java, minSize >= 1000) ---
Match: src/main/App.java

--- SearchVisitor (all files, minSize >= 10000) ---
Match: docs/architecture.png
Match: docs/specification.pdf

--- StatisticsVisitor ---
Total files: 8 | Total directories: 7 | Maximum tree depth: 3

--- Verification of Reusability (No State Leakage) ---
Re-run size visitor: 71570 bytes | Re-run statistics: files: 8, dirs: 7, depth: 3
Caught cycle detection: Cycle detected: cannot add ancestor as child
Caught negative size: File size cannot be negative: -10
```

7. Automated Verification Summary (JUnit 5 Suite)

Test Class: kz.aitu.se2523.visitor.FileSystemVisitorTest | Runner: Maven Surefire 3.2.5 (mvn clean test)

Check	Test Method Name	Automated Verification Target	Status
T1	`t1_bothElementTypesReachCorrectOverloadedVisitMethod`	Double dispatch routes FileNode and DirectoryNode correctly	**PASS**
T2	`t2_sizeAccumulationAcrossMultipleNestingLevelsIsCorrect`	Cumulative size across 3 nesting levels (7000 bytes)	**PASS**
T3	`t3_searchCorrectlyCombinesBothConfigurableFilterCriteria`	Extension (.java) combined with minimum size threshold	**PASS**
T4	`t4_searchProducesCorrectDeterministicRootRelativePaths`	Root-relative paths with '/' separator, lexicographically sorted	**PASS**
T5	`t5_statisticsAreCorrectForNestingCountingAndMaxDepth`	File count (3), directory count (3), and max depth (3) accurate	**PASS**
T6	`t6_emptyStructuresAndNoMatchSearchCasesAreHandled`	Empty directory handling and empty search result sets verified	**PASS**
T7	`t7_repeatExecutionsDoNotLeakOrMultiplyAccumulatedState`	Repeated runs do not leak or double state (reusability verified)	**PASS**
T8	`t8_invalidTreeMutationsAreRejectedWhileValidOnesSucceed`	Rejecting self-, duplicate siblings, multi-parent & cycles	**PASS**

Test Execution Result: 8 Tests Run | 0 Failures | 0 Errors | 0 Skipped | Time: 0.021s [100% SUCCESS]

8. Academic References & Defense Preparation

Defense Protocol Checklist (Prepared Responses):

Step 1 (Double Dispatch Trace): Client calls element.accept(visitor) (polymorphism on Element). Inside node, visitor.visit(this) is invoked where 'this' has exact compile-time static type, dispatching to visit(FileNode) or visit(DirectoryNode).

Step 2 (Automated Test Suite): Execute 'mvn clean test' or './mvnw test' from project root; all 8 test cases pass in 0.021s verifying double dispatch, size calculation, search filtering, statistics, idempotency, and invariant defenses.

Step 3 (Configuration Mutation): SearchVisitor supports dynamic predicate or size changes; StatisticsVisitor supports arbitrary depth and tree structures without requiring modifications to FileNode or DirectoryNode.

Step 4 (Architectural Trade-off): Adding new operations requires only a new Visitor implementation (OCP compliant). Adding new elements requires modifying the Visitor interface and updating all existing visitor classes.

Academic Literature:

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
2. Freeman, E., & Robson, E. (2021). Head First Design Patterns (2nd ed.), Ch. 11: Visitor Pattern. O'Reilly Media.
3. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
4. ShP-2216 Lecture 9: Behavioral Patterns - Visitor, Composite Structures, and Double Dispatch. Astana IT University LMS.

Submission Link: https://github.com/M1ska2021/Assignment6_Visitor